

Dokumentacion Teknik dhe Akademik për Ndërtimin e Modelit FARSH Law Assistant

Nëntitull

Projektim, trajnim, testim dhe implementim i një asistenti të inteligjencës artificiale në gjuhën shqipe, të specializuar mbi dokumentin "Manuali Kombëtar i Operacioneve të Kërkim-Shpëtimit të FARSH".

Abstrakt

Ky dokument paraqet procesin teknik dhe metodologjik të ndjekur për ndërtimin e një modeli të specializuar të inteligjencës artificiale në gjuhën shqipe, të trajnuar mbi dokumentin "**Manuali Kombëtar i Operacioneve të Kërkim-Shpëtimit të FARSH**". Projekti synon zhvillimin e një asistenti të aftë për të gjeneruar përgjigje të sakta, të kontrolluara dhe të kufizuara brenda domenit të dokumentit burimor. Për realizimin e këtij qëllimi është përdorur një pipeline i plotë, i përbërë nga përgatitja e dataset-it, konfigurimi i ambientit të zhvillimit, fine-tuning i modelit bazë me teknikën QLoRA, testimi i adapter-it, vendosja e mekanizmave të kontrollit të sjelljes, integrimi i një shtrese shërbimi me FastAPI, bashkimi i modelit, optimizimi për inferencë lokale dhe deploy-i në Ollama. Në versionin e përmirësuar të projektit, dataset-i është zgjeruar në **1000 shembuj**, ndërsa sistemi final arrin të japë përgjigje të sakta për pyetjet që mbulohen nga dataset-i dhe të kthejë në mënyrë të qëndrueshme një përgjigje refuzuese për pyetjet që janë jashtë fushës së studimit. Kjo e bën zgjidhjen më të kontrollueshme, më të besueshme dhe më të përshtatshme për përdorim praktik.

Fjalë kyçe

Inteligjencë artificiale, fine-tuning, QLoRA, Mistral, FastAPI, dataset JSONL, guardrails, GGUF, Ollama, gjuhë shqipe, model i specializuar.

1. Hyrje

Zhvillimi i modeleve gjuhësore të mëdha ka krijuar mundësi të reja për ndërtimin e sistemeve inteligjente të afta për të kuptuar dhe gjeneruar tekst në gjuhë natyrore. Megjithatë, modelet e përgjithshme shpesh nuk garantojnë saktësinë dhe kontrollueshmërinë e nevojshme kur përdoren në fusha të specializuara ose kur kërkohet mbështetje e drejtpërdrejtë në dokumentacion të caktuar.

Në këtë kontekst, ky projekt fokusohet në ndërtimin e një asistenti AI në gjuhën shqipe, të specializuar mbi dokumentin "**Manuali Kombëtar i Operacioneve të Kërkim-Shpëtimit të FARSH**". Qasja e ndjekur nuk synon vetëm përdorimin e një modeli ekzistues, por përshtatjen e tij përmes fine-tuning dhe kontrollin e sjelljes përmes një shtrese ndërmjetëse shërbimi, në mënyrë që përgjigjet e gjeneruara të jenë sa më të lidhura me përmbajtjen e dokumentit dhe të shmanget prodhimi i informacionit jashtë fushës së studimit.

Ky dokument ka karakter teknik dhe akademik. Ai përshkruan jo vetëm hapat e realizimit të projektit, por edhe funksionin, rëndësinë dhe rezultatet e secilës fazë në ndërtimin e sistemit përfundimtar.

2. Qëllimi i projektit

Qëllimi kryesor i projektit është ndërtimi i një asistenti të inteligjencës artificiale në gjuhën shqipe, i aftë të përgjigjet në mënyrë të kontrolluar dhe të specializuar mbi informacionin e përmbajtur në dokumentin e FARSH për operacionet e kërkim-shpëtimit.

Në aspekt funksional, projekti synon:

- të përshtatë një model bazë gjuhësor për një domen të caktuar njohurish
- të mundësojë gjenerimin e përgjigjeve të mbështetura në dokumentacion burimor
- të kufizojë sjelljen e modelit jashtë domenit të tij të trajnimit
- të kombinojë modelin gjuhësor me një shtresë logjike kontrolli përmes **FastAPI**
- të prodhojë një model të përdorshëm lokalisht përmes një pipeline-i të plotë teknik

3. Objektivat

Për realizimin e qëllimit kryesor janë përcaktuar këto objektiva specifike:

- ndërtimi dhe zgjerimi i një dataset-i të strukturuar në format **JSONL**
- përgatitja e një ambienti të kontrolluar zhvillimi për trajnim dhe testim
- realizimi i fine-tuning të modelit bazë me teknikën **QLoRA**
- vlerësimi i modelit të trajnuar përmes pyetjeve brenda dhe jashtë domenit
- implementimi i mekanizmave kufizues për kontrollin e sjelljes së modelit
- ndërtimi i një **API** shërbimi me **FastAPI** për menaxhimin e logjikës së përgjigjeve
- bashkimi i adapter-it me modelin bazë për krijimin e një artefakti të vetëm
- konvertimi dhe optimizimi i modelit për inferencë lokale
- deploy-i i modelit final në **Ollama**

4. Metodologjia e ndjekur

Metodologjia e projektit është ndërtuar mbi një qasje inxhinierike dhe eksperimentale, në të cilën secila fazë prodhon një rezultat të ndërmjetëm që përdoret si hyrje për fazën pasuese. Procesi është organizuar në mënyrë lineare dhe të kontrolluar, duke filluar nga përgatitja e të dhënave dhe duke përfunduar te përdorimi i modelit për inferencë lokale përmes një shërbimi **API**.

Paraqitje e përgjithshme e pipeline-it

flowchart LR

```
A[Dataset JSONL] --> B[Trajnimi me QLoRA]
B --> C[LoRA Adapter]
C --> D[Testimi i adapter-it]
D --> E[Guardrails]
E --> F[FastAPI Service Layer]
F --> G[Merge i modelit]
```

```
G --> H[Konvertimi ne GGUF]
H --> I[Quantization Q4_K_M]
I --> J[Deploy ne Ollama]
```

Kjo rrjedhë paraqet transformimin e dokumentit burimor në një sistem të përdorshëm lokalisht. Çdo fazë kontribuon drejtpërdrejt në saktësinë, kontrollueshmërinë, efikasitetin dhe praktikueshmërinë e zgjidhjes finale.

5. Përgatitja e dataset-it

File

`dataset_ksh_farsh.jsonl`

Përshkrimi i fazës

Faza e parë konsiston në ndërtimin dhe ristrukturimin e dataset-it që përdoret për fine-tuning. Dataset-i është organizuar në formatin `JSONL`, ku çdo rresht përfaqëson një shembull trajnimi në formën e një ndërveprimi ndërmjet përdoruesit dhe asistentit.

Funksioni i kësaj faze

Kjo fazë ka funksionin themelor të transformimit të përmbajtjes së dokumentit burimor në një formë të kuptueshme për modelin gjuhësor. Përmes dataset-it, modeli ekspozohet ndaj:

- terminologjisë së specializuar të operacioneve SAR
- pyetjeve përfaqësuese që lidhen me dokumentin
- formulimit të përgjigjeve në gjuhën shqipe
- kufijve konceptualë të domenit ku modeli duhet të operojë

Formati i përdorur

Secili shembull është i ndërtuar me fushën `messages`, e cila përmban të paktën një mesazh me rolin `user` dhe një mesazh me rolin `assistant`.

Shembull i strukturës së dataset-it

```
{"messages": [{"role": "user", "content": "Çfarë është RCC?"}, {"role": "assistant", "content": "RCC është Qendra e Koordinim-Shpëtimit që kontrollon dhe koordinon incidentet SAR."}]}
```

Rezultati i fazës

Në versionin e përditësuar të projektit, dataset-i është zgjeruar dhe rregulluar në **1000 shembuj pyetje-përgjigje**. Ky zgjerim ka rritur mbulimin e pyetjeve, ka përmirësuar saktësinë e përgjigjeve dhe ka krijuar

bazën për një sjellje më të qëndrueshme të sistemit gjatë përdorimit real.

6. Përgatitja e ambientit të zhvillimit

Përshkrimi i fazës

Përpara fillimit të trajnimit, është përgatitur një ambient i izoluar zhvillimi përmes përdorimit të mjedisit virtual **venv**. Kjo qasje mundëson kontroll më të lartë mbi dependencat dhe minimizon ndikimin e konfigurimeve ekzistuese të sistemit.

Funksioni i kësaj faze

Kjo fazë siguron:

- instalim të kontrolluar të librarive të nevojshme
- shmangie të konflikteve ndërmjet versioneve të paketave
- riprodhueshmëri të procesit teknik
- stabilitet gjatë fazave të trajnimit, testimit dhe optimizimit

Aktivizimi i mjedisit virtual

```
.venv\Scripts\activate
```

Kalimi në folderin e trajnimit

```
cd training
```

Roli i folderit të trajnimit

Folderi i trajnimit përmban skriptet kryesore të projektit, të përdorura për trajnim, testim, shërbimin **API**, bashkim të modelit dhe ruajtje të output-eve për secilën fazë të pipeline-it.

7. Trajnimi i modelit

File

train_law_ai.py

Përshkrimi i fazës

Trajnimi i modelit synon përshtatjen e modelit bazë **Mistral-7B-Instruct-v0.3** me njohuritë e përfshira në dokumentin e FARSH. Në këtë projekt është përdorur teknika **QLoRA**, e cila mundëson fine-tuning efikas të

modeleve të mëdha duke reduktuar kostot llogaritëse dhe konsumin e memories.

Funksioni i skriptit

Skripti `train_law_ai.py` realizon këto detyra kryesore:

- ngarkimin e dataset-it
- filtrimin e shembujve jo-validë
- formatimin e shembujve sipas strukturës së kërkuar nga modeli
- ngarkimin e modelit bazë me `4-bit quantization`
- përgatitjen e modelit për `k-bit training`
- konfigurimin e parametrave `LoRA`
- ekzekutimin e trajnimit
- ruajtjen e adapter-it dhe tokenizer-it

Arsyeja e përdorimit të QLoRA

Teknika `QLoRA` është zgjedhur për shkak se lejon përshtatjen e modeleve të mëdha edhe në mjedise me resurse të kufizuara, duke ruajtur performancë të mirë në detyra të specializuara.

Parametrat teknikë kryesorë

- model bazë: `mistralai/Mistral-7B-Instruct-v0.3`
- quantization: `4-bit`
- precision: `BF16`
- metodë adaptimi: `LoRA`
- batch size për pajisje: `1`
- gradient accumulation steps: `8`
- numri i epokave: `5`

Komanda e ekzekutimit

```
(.venv) PS C:\Users\bajra\Desktop\fine-tuning-docs\training> python  
train_law_ai.py
```

Kodi i plotë i file-it

```
import os  
os.environ["ACCELERATE_MIXED_PRECISION"] = "bf16"  
os.environ["ACCELERATE_USE_BF16"] = "1"  
  
from datasets import load_dataset  
from transformers import (  
    AutoModelForCausalLM,  
    AutoTokenizer,  
    BitsAndBytesConfig,  
)
```

```

from peft import LoraConfig, prepare_model_for_kbit_training
from trl import SFTTrainer, SFTConfig
import torch

BASE_MODEL = "mistralai/Mistral-7B-Instruct-v0.3"
DATASET_FILE = r"C:\Users\bajra\Desktop\fine-tuning-docs\dataset\dataset_ksh_farsh.jsonl"
OUTPUT_DIR = r"C:\Users\bajra\Desktop\fine-tuning-docs\training\outputs\albanian-law-assistant-helper"

def format_example(example):
    messages = example.get("messages", [])

    if not messages or len(messages) < 2:
        return ""

    user_msg = ""
    assistant_msg = ""

    for msg in messages:
        role = msg.get("role", "").strip()
        content = msg.get("content", "").strip()

        if not content:
            continue

        if role == "user":
            user_msg = content
        elif role == "assistant":
            assistant_msg = content

    if not user_msg or not assistant_msg:
        return ""

    return f"<s>[INST] {user_msg} [/INST] {assistant_msg}</s>"

dataset = load_dataset("json", data_files=DATASET_FILE, split="train")
dataset = dataset.filter(lambda x: len(format_example(x)) > 0)

bnb_config = BitsAndBytesConfig(
    load_in_4bit=True,
    bnb_4bit_quant_type="nf4",
    bnb_4bit_compute_dtype=torch.bfloat16,
    bnb_4bit_use_double_quant=True,
)

tokenizer = AutoTokenizer.from_pretrained(BASE_MODEL)
if tokenizer.pad_token is None:
    tokenizer.pad_token = tokenizer.eos_token
tokenizer.padding_side = "right"

print("Dataset size after filtering:", len(dataset))

```

```

print("CUDA available:", torch.cuda.is_available())
if torch.cuda.is_available():
    print("GPU:", torch.cuda.get_device_name(0))
    print("BF16 supported:", torch.cuda.is_bf16_supported())
else:
    raise RuntimeError("CUDA nuk është aktive.")

model = AutoModelForCausalLM.from_pretrained(
    BASE_MODEL,
    quantization_config=bnb_config,
    device_map="auto",
    dtype=torch.bfloat16,
)

model.config.use_cache = False
model = prepare_model_for_kbit_training(
    model,
    use_gradient_checkpointing=True
)

peft_config = LoraConfig(
    r=16,
    lora_alpha=32,
    lora_dropout=0.05,
    bias="none",
    task_type="CAUSAL_LM",
    target_modules=[
        "q_proj",
        "k_proj",
        "v_proj",
        "o_proj",
        "gate_proj",
        "up_proj",
        "down_proj",
    ],
)

training_args = SFTConfig(
    output_dir=OUTPUT_DIR,
    per_device_train_batch_size=1,
    gradient_accumulation_steps=8,
    num_train_epochs=5,
    learning_rate=2e-4,
    logging_steps=5,
    save_strategy="epoch",
    save_total_limit=2,
    report_to="none",
    gradient_checkpointing=True,
    optim="paged_adamw_8bit",
    fp16=False,
    bf16=True,
    max_grad_norm=1.0,
    max_steps=-1,
    warmup_ratio=0.03,

```

```
lr_scheduler_type="cosine",
packing=False,
)

trainer = SFTTrainer(
    model=model,
    train_dataset=dataset,
    args=training_args,
    peft_config=peft_config,
    formatting_func=format_example,
)

trainer.train()
trainer.model.save_pretrained(OUTPUT_DIR)
tokenizer.save_pretrained(OUTPUT_DIR)

print(f"Training complete. Adapter saved to: {OUTPUT_DIR}")
```

Rezultati i fazës

Rezultati i drejtpërdrejtë i kësaj faze është krijimi i adapter-it **LoRA**, i ruajtur në folderin e output-it, i cili më pas përdoret për inferencë dhe për bashkim me modelin bazë.

8. Testimi i adapter-it

File

`test_adapter_law.py`

Përshkrimi i fazës

Pas përfundimit të trajnimit, është e nevojshme të verifikohet nëse modeli i trajnuar prodhon përgjigje të sakta, të qëndrueshme dhe të kufizuara sipas dokumentit burimor. Kjo fazë përbën kontrollin funksional të modelit në nivel inference.

Funksioni i skriptit

Skripti `test_adapter_law.py` realizon këto funksione:

- ngarkon modelin bazë dhe adapter-in **LoRA**
- ndërton prompt-in e testimit
- vendos rregulla kufizuese për sjelljen e modelit
- teston modelin me pyetje të ndryshme
- verifikon reagimin e modelit për pyetje brenda dhe jashtë domenit

Përfundimi i testimit

Në gjendjen aktuale të projektit, testimi ka treguar se pyetjet që janë pjesë e dataset-it marrin përgjigje të sakta, ndërsa pyetjet që janë jashtë fushës së studimit marrin një përgjigje refuzuese të qëndrueshme. Kjo tregon një përmirësim domethënës krahasuar me fazat më të hershme të projektit.

9. Guardrails dhe kontrolli i sjelljes së modelit

Përshkrimi i fazës

Guardrails përfaqësojnë mekanizmat kufizues që synojnë kontrollin e sjelljes së modelit gjatë inferencës. Në këtë projekt, ato janë ndërtuar përmes udhëzimeve në prompt, përmes filtrimit të pyetjeve sipas domenit dhe përmes një përgjigjeje standarde refuzimi për pyetjet jashtë fushës së studimit.

Funksioni i guardrails

Këto mekanizma shërbejnë për:

- kufizimin e modelit brenda domenit të dokumentit FARSH
- reduktimin e gjasave për hallucination
- rritjen e besueshmërisë së përgjigjeve
- standardizimin e sjelljes kur pyetja nuk i përket fushës së studimit

Default response

Kjo pyetje është jashtë tematikës së studimit.

10. Implementimi i shtresës së shërbimit me FastAPI

File

`api_server.py`

Përshkrimi i fazës

Përveç vetë modelit të trajnuar, në projekt është ndërtuar edhe një shtresë logjike aplikative përmes **FastAPI**. Kjo shtresë shërben si ndërmjetëse midis përdoruesit dhe modelit, duke kontrolluar mënyrën se si trajtohen pyetjet dhe duke vendosur rregulla shtesë mbi përgjigjet.

Funksioni i `api_server.py`

Ky file realizon disa funksione thelbësore:

- ngarkon dataset-in në memorie dhe krijon një hartë pyetje-përgjigje
- normalizon tekstin për krahasim më të qëndrueshëm
- kontrollon nëse pyetja i përket domenit SAR/FARSH
- kthen menjëherë përgjigjen e saktë nëse pyetja ekziston saktësisht në dataset
- kërkon pyetje të ngjashme përmes **fuzzy matching**

- përdor modelin në **Ollama** vetëm kur është e nevojshme
- kthen automatikisht një përgjigje standarde për pyetjet jashtë fushës së studimit
- ofron endpoint-e **REST** dhe endpoint-e në format të ngjashëm me **OpenAI Chat Completions**

Logjika funksionale e shërbimit

Shtresa **FastAPI** ndjek këtë rend logjik:

1. Pyetja normalizohet.
2. Kontrollohet nëse pyetja ekziston saktësisht në dataset.
3. Nëse nuk ekziston, vlerësohet nëse është brenda fushës së studimit.
4. Nëse është jashtë fushës, kthehet menjëherë përgjigja refuzuese.
5. Nëse është brenda fushës, kërkohet një përputhje e afërt në dataset.
6. Vetëm nëse nuk gjendet përputhje, pyetja i dërgohet modelit në **Ollama**.

Vlera e kësaj zgjidhjeje

Përdorimi i **FastAPI** e ka përmirësuar ndjeshëm kontrollueshmërinë e sistemit, sepse përgjigja nuk lihet plotësisht në dorën e modelit gjuhësor. Përkundrazi, modeli vendoset brenda një pipeline-i logjik ku përgjigjet e sakta nga dataset-i kanë përparësi, ndërsa pyetjet jashtë tematikës filtrohen në mënyrë të qartë.

Kodi i përdorur

```
from fastapi import FastAPI, HTTPException
from fastapi.responses import JSONResponse
from fastapi.middleware.cors import CORSMiddleware
from pydantic import BaseModel
from pathlib import Path
from difflib import get_close_matches, SequenceMatcher
import subprocess
import json
import re
import time
import uuid

app = FastAPI(
    title="Hybrid QA API",
    openapi_url="/v1/openapi.json"
)

app.add_middleware(
    CORSMiddleware,
    allow_origins=["*"],
    allow_credentials=True,
    allow_methods=["*"],
    allow_headers=["*"],
)

DATASET_PATH = Path(r"C:\Users\bajra\Desktop\fine-tuning-docs\dataset\dataset_ksh_farsh.jsonl")
```

```

OLLAMA_MODEL = "law-assistant"
REFORM_MODEL = "mistral"
PUBLIC_MODEL_ID = "law-assistant-api"

SAR_KEYWORDS = [
    "sar", "kërkim", "shpëtim", "kerkim", "shpetim",
    "rcc", "smc", "incerfa", "alerfa", "detresfa",
    "medevac", "plb", "farsh",
    "operacion", "operacione", "operacioni",
    "faz", "faza", "fazat", "phase",
    "emergjenc", "manual", "koordinim", "rescue",
    "search and rescue", "alarm", "distress",
    "mision", "avion", "helikopter", "detar",
    "ajror", "tokësor", "tokesor", "plb"
]

OUT_OF_SCOPE_MESSAGE = "Kjo pyetje është jashtë tematikës së studimit."

def normalize_text(text: str) -> str:
    text = text.strip().lower()
    text = re.sub(r"\s+", " ", text)
    text = text.replace("ç", "c").replace("ë", "e")
    return text

def is_in_scope(text: str) -> bool:
    normalized = normalize_text(text)

    if any(keyword in normalized for keyword in SAR_KEYWORDS):
        return True

    domain_terms = {"rcc", "smc", "plb", "incerfa", "alerfa", "detresfa",
"medevac"}
    words = set(re.findall(r"[a-zA-ZçëÇË]+", normalized))
    if words.intersection(domain_terms):
        return True

    return False

def load_dataset(path: Path) -> dict[str, str]:
    qa_map: dict[str, str] = {}

    with path.open("r", encoding="utf-8") as f:
        for line in f:
            line = line.strip()
            if not line:
                continue

            obj = json.loads(line)
            messages = obj.get("messages", [])
            if len(messages) < 2:
                continue

```

```
        user_msg = messages[0]
        assistant_msg = messages[1]

        if user_msg.get("role") != "user" or assistant_msg.get("role") !=
"assistant":
            continue

        question = user_msg.get("content", "").strip()
        answer = assistant_msg.get("content", "").strip()

        if question and answer:
            qa_map[normalize_text(question)] = answer

    return qa_map

QA_MAP = load_dataset(DATASET_PATH)

def find_similar(question: str, qa_map: dict[str, str]) -> str | None:
    if len(question.split()) < 3:
        return None

    matches = get_close_matches(question, qa_map.keys(), n=1, cutoff=0.75)
    if not matches:
        return None

    matched_question = matches[0]
    similarity_ratio = SequenceMatcher(None, question, matched_question).ratio()

    if similarity_ratio < 0.75:
        return None

    return qa_map[matched_question]

def clean_answer(answer: str) -> str:
    answer = re.sub(r"\s+", " ", answer).strip()
    answer = re.sub(r"\s+([.,!?:])", r"\1", answer)
    answer = re.sub(r"([.!?]){2,}", r"\1", answer)

    if not answer or len(answer) < 5:
        return "Më vjen keq, nuk arrita të jap një përgjigje të saktë."

    if not answer.endswith((".", "!", "?")):
        answer += "."

    return answer

def build_out_of_scope_answer() -> str:
    return OUT_OF_SCOPE_MESSAGE
```

```
def run_ollama(model_name: str, prompt: str) -> str:
    result = subprocess.run(
        ["ollama", "run", model_name, prompt],
        capture_output=True,
        text=True,
        encoding="utf-8",
        errors="ignore"
    )

    if result.returncode != 0:
        raise RuntimeError(result.stderr.strip() or f"Ollama failed for model {model_name}")

    return result.stdout.strip()

def improve_albanian(text: str) -> str:
    system_prompt = (
        "Riformulo tekstin në shqip standarde. "
        "Korrigjo gabimet gramatikore dhe stilistike. "
        "Bëje tekstin natyral, profesional dhe të rrjedhshëm. "
        "Mos ndrysho kuptimin. "
        "Mos shto informacion të ri. "
        "Kthe vetëm versionin e përmirësuar të tekstit."
    )

    full_prompt = f"{system_prompt}\n\nTeksti:\n{text}"

    try:
        improved = run_ollama(REFORM_MODEL, full_prompt)
        improved = clean_answer(improved)
        return improved if improved else text
    except Exception:
        return text

def ask_ollama(prompt: str) -> str:
    system_prompt = (
        "Ti je një asistent që përgjigjet vetëm për tematikën SAR/FARSH. "
        "Përgjigju vetëm në shqip standarde. "
        "Mos përdor anglisht. "
        "Mos jep përkthime. "
        "Mos shpik informacion. "
        f"Nëse pyetja është jashtë tematikës, kthe saktë vetëm këtë fjali: {OUT_OF_SCOPE_MESSAGE} "
        "Nëse pyetja është brenda tematikës, jep përgjigje të saktë dhe të qartë."
    )

    full_prompt = f"{system_prompt}\n\nPyetja: {prompt}"
    raw_answer = run_ollama(OLLAMA_MODEL, full_prompt)
    raw_answer = clean_answer(raw_answer)

    if normalize_text(raw_answer) == normalize_text(OUT_OF_SCOPE_MESSAGE):
```

```
        return OUT_OF_SCOPE_MESSAGE

    return raw_answer

class SimpleChatRequest(BaseModel):
    prompt: str

class ChatMessage(BaseModel):
    role: str
    content: str

class ChatCompletionsRequest(BaseModel):
    model: str | None = None
    messages: list[ChatMessage]
    temperature: float | None = None

@app.get("/")
def root():
    return {
        "status": "ok",
        "message": "Hybrid QA API is running",
        "model": PUBLIC_MODEL_ID,
    }

@app.get("/health")
def health():
    return {
        "status": "ok",
        "dataset_entries": len(QA_MAP),
        "ollama_model": OLLAMA_MODEL,
        "reform_model": REFORM_MODEL,
        "public_model": PUBLIC_MODEL_ID,
    }

@app.get("/v1/models")
def list_models():
    return JSONResponse(
        content={
            "object": "list",
            "data": [
                {
                    "id": PUBLIC_MODEL_ID,
                    "object": "model",
                    "created": int(time.time()),
                    "owned_by": "local",
                }
            ],
        },
    )
```

```
        media_type="application/json; charset=utf-8",
    )

@app.post("/chat")
def simple_chat(req: SimpleChatRequest):
    question = req.prompt.strip()
    normalized = normalize_text(question)

    if normalized in QA_MAP:
        return JSONResponse(
            content={"response": QA_MAP[normalized], "source": "exact"},
            media_type="application/json; charset=utf-8",
        )

    if not is_in_scope(question):
        answer = build_out_of_scope_answer()
        return JSONResponse(
            content={"response": answer, "source": "out_of_scope"},
            media_type="application/json; charset=utf-8",
        )

    similar = find_similar(normalized, QA_MAP)
    if similar:
        return JSONResponse(
            content={"response": similar, "source": "fuzzy"},
            media_type="application/json; charset=utf-8",
        )

    answer = ask_ollama(question)
    return JSONResponse(
        content={"response": answer, "source": "ollama"},
        media_type="application/json; charset=utf-8",
    )

@app.post("/v1/chat/completions")
def chat_completions(req: ChatCompletionsRequest):
    if not req.messages:
        raise HTTPException(status_code=400, detail="messages list is empty")

    user_messages = [
        m.content.strip()
        for m in req.messages
        if m.role == "user" and m.content.strip()
    ]

    if not user_messages:
        raise HTTPException(status_code=400, detail="no user message found")

    last_user_message = user_messages[-1]
    normalized = normalize_text(last_user_message)

    source = "ollama"
```

```
if normalized in QA_MAP:
    answer = QA_MAP[normalized]
    source = "exact"
else:
    if not is_in_scope(last_user_message):
        answer = build_out_of_scope_answer()
        source = "out_of_scope"
    else:
        similar = find_similar(normalized, QA_MAP)
        if similar:
            answer = similar
            source = "fuzzy"
        else:
            answer = ask_ollama(last_user_message)
            source = "ollama"

now = int(time.time())
response_payload = {
    "id": f"chatcpl-{uuid.uuid4().hex}",
    "object": "chat.completion",
    "created": now,
    "model": PUBLIC_MODEL_ID,
    "choices": [
        {
            "index": 0,
            "message": {
                "role": "assistant",
                "content": answer,
            },
            "finish_reason": "stop",
        }
    ],
    "usage": {
        "prompt_tokens": 0,
        "completion_tokens": 0,
        "total_tokens": 0,
    },
    "source": source,
}

return JsonResponse(
    content=response_payload,
    media_type="application/json; charset=utf-8",
)
```

11. Merge i modelit

File

`merge_farsh_law_model.py`

Përshkrimi i fazës

Pas trajnimit, produkti kryesor është adapter-i **LoRA**. Megjithatë, për ta bërë modelin më të lehtë për përdorim në fazat pasuese, adapter-i bashkohet me modelin bazë në një artefakt të vetëm.

Funksioni i skriptit

Skripti `merge_farsh_law_model.py` realizon:

- ngarkimin e modelit bazë
- ngarkimin e tokenizer-it
- ngarkimin e adapter-it **LoRA**
- bashkimin e adapter-it me modelin bazë
- ruajtjen e modelit të bashkuar në një strukturë të re

Rëndësia e kësaj faze

Kjo fazë është e rëndësishme sepse krijon një version të vetëm të modelit, i cili mund të përdoret më lehtë për konvertim, quantization dhe deploy.

12. Konvertimi i modelit në format GGUF F16

Përshkrimi i fazës

Pasi modeli është bashkuar, ai konvertohet në formatin **GGUF**, i cili është më i përshtatshëm për inferencë lokale dhe për integrim me mjedise si **llama.cpp**.

Funksioni i kësaj faze

Kjo fazë realizon:

- transformimin e modelit Hugging Face në një format alternativ inferencë
- përgatitjen e modelit për quantization
- krijimin e një artefakti më praktik për përdorim lokal

13. Quantization i modelit në **Q4_K_M**

Përshkrimi i fazës

Në këtë fazë aplikohet quantization për të zvogëluar madhësinë e modelit dhe për të rritur efikasitetin e inferencës lokale.

Funksioni i kësaj faze

Quantization shërben për:

- reduktimin e përdorimit të memories
- përmirësimin e shpejtësisë së ekzekutimit
- rritjen e praktikueshmërisë së modelit në mjedise lokale
- ruajtjen e një kompromisi të arsyeshëm ndërmjet madhësisë dhe performancës

Formati **Q4_K_M** është përzgjedhur si një zgjidhje e balancuar për inferencë efikase.

14. Krijimi i **Modelfile** për Ollama

Përshkrimi i fazës

Për ta bërë modelin të përdorshëm në **Ollama**, krijohet një **Modelfile**, i cili përkufizon burimin e modelit, template-in e prompt-it, parametrat e inferencës dhe rregullat kryesore të sjelljes.

Funksioni i **Modelfile**

Ky file ka funksion konfigurues dhe përcakton:

- artefaktin që do të përdoret si model
 - strukturën e prompt-it
 - parametrat e gjenerimit
 - rregullat kryesore të sjelljes së modelit
-

15. Krijimi dhe deploy-i i modelit në Ollama

Përshkrimi i fazës

Faza përfundimtare konsiston në krijimin e modelit final në **Ollama**, duke e bërë atë të përdorshëm për inferencë lokale dhe për ndërveprim të drejtpërdrejtë me përdoruesin.

Funksioni i kësaj faze

Kjo fazë materializon të gjithë pipeline-in e projektit në një produkt funksional, i cili mund të përdoret për pyetje reale mbi përmbajtjen e dokumentit të trajnuar.

16. Rezultatet dhe diskutimi

Rezultatet e projektit tregojnë se pipeline-i teknik është realizuar me sukses dhe ka prodhuar një sistem funksional për përdorim lokal në gjuhën shqipe.

Rezultatet kryesore

- dataset-i është ristrukturuar dhe zgjeruar në **1000 shembuj**
- pyetjet që janë pjesë e dataset-it marrin përgjigje të sakta

- pyetjet jashtë fushës së studimit marrin përgjigjen standarde refuzuese
- shtresa **FastAPI** kontrollon dhe stabilizon rrjedhën e përgjigjes
- modeli final mbetet i përdorshëm lokalisht përmes **Ollama**

Diskutim mbi përmirësimin

Krahasuar me fazat e mëparshme të projektit, versioni aktual paraqet një përmirësim të dukshëm në kontrollin e sjelljes së sistemit. Kjo vjen jo vetëm nga zgjerimi i dataset-it, por edhe nga kombinimi i modelit të trajnuar me një shtresë logjike aplikative që menaxhon verifikimin e pyetjeve, përputhjen me dataset-in dhe filtrimin e pyetjeve jashtë domenit.

Ky kombinim e kthen sistemin nga një model gjuhësor i trajnuar në një zgjidhje hibride më të besueshme dhe më të kontrollueshme.

17. Konkluzione

Nga analiza e të gjithë procesit rezulton se ndërtimi i një modeli të specializuar në gjuhën shqipe mbi bazën e një dokumenti të vetëm është teknikisht i realizueshëm dhe praktik, sidomos kur kombinohen teknika si **QLoRA**, mekanizma kufizues për kontrollin e sjelljes dhe një shtresë aplikative si **FastAPI** për menaxhimin e logjikës së përgjigjeve.

Ky projekt dëshmon se:

- modelet bazë gjuhësore mund të përshtaten në mënyrë efikase për detyra të specializuara
- zgjerimi i dataset-it ndikon drejtpërdrejt në saktësinë e përgjigjeve
- një shtresë aplikative e ndërmjetme rrit ndjeshëm kontrollueshmërinë e sistemit
- optimizimet si **GGUF** dhe **Q4_K_M** e bëjnë modelin të përdorshëm në mjedise lokale

Në aspekt akademik, projekti përfaqëson një shembull konkret të ndërtimit të një sistemi hibrid që kombinon fine-tuning, logjikë aplikative dhe deploy lokal për një fushë të specializuar në gjuhën shqipe.

18. Rekomandime për punë të ardhshme

Për përmirësimin e mëtejshëm të sistemit rekomandohen këto drejtime:

- zgjerimi i mëtejshëm i dataset-it me pyetje më të larmishme
- shtimi i testeve automatike për endpoint-et e **FastAPI**
- ruajtja e statistikave të pyetjeve reale për analizë të mëvonshme
- integrimi i një mekanizmi **RAG** për dokumente shtesë
- përmirësimi i metrikave të vlerësimit për saktësi dhe robustesë

19. Përmbledhje e rrjedhës së plotë të projektit

1. U ndërtua dhe u rregullua dataset-i **dataset_ksh_farsh.jsonl**.

2. Dataset-i u zgjerua në **1000** shembuj pyetje-përgjigje.
3. U përgatit ambienti virtual i zhvillimit dhe dependencat e projektit.
4. U ekzekutua **train_law_ai.py** për fine-tuning të modelit bazë.
5. U krijua adapter-i **LoRA**.
6. U testua modeli me **test_adapter_law.py**.
7. U vendosën guardrails për kontrollin e sjelljes.
8. U ndërtua shtresa e shërbimit **api_server.py** me **FastAPI**.
9. U realizua logjika e kontrollit për pyetje ekzakte, pyetje të ngjashme, pyetje brenda domenit dhe pyetje jashtë domenit.
10. U realizua merge i adapter-it me modelin bazë.
11. U konvertua modeli në format **GGUF F16**.
12. U aplikua quantization në formatin **Q4_K_M**.
13. U krijua **Modelfile** për integrim në **Ollama**.
14. U krijua modeli final për përdorim lokal.